

Day 29 – Introduction to Docker

Task 1: What is Docker?

1. What is a Container and Why Do We Need Them?

A **container** is a lightweight package that contains:

- Application code
- Runtime environment
- Libraries
- Dependencies
- Configuration files

Containers ensure that an application runs the same way on any machine.

Why Do We Need Containers?

Before containers:

- Applications worked on one machine but failed on another.
- Different software versions caused compatibility issues.
- Setting up environments was time-consuming.

Containers solve these problems by packaging everything needed to run an application.

2. Containers vs Virtual Machines

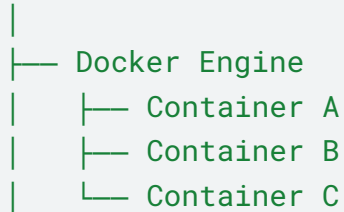
Feature	Containers	Virtual Machines
OS	Share host OS kernel	Each VM has its own OS
Size	Small (MBs)	Large (GBs)
Startup Time	Seconds	Minutes
Performance	Near-native	More overhead
Resource Usage	Low	High
Isolation	Process-level	Hardware-level

Diagram

None

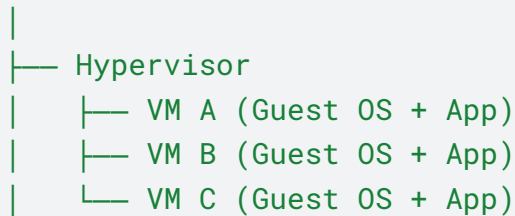
Containers

Host OS



Virtual Machines

Host OS



3. Docker Architecture

Docker consists of:

Docker Client

The command-line tool.

Example:

None

```
docker run nginx
```

Docker Daemon

Background service that:

- Builds images
 - Runs containers
 - Manages networks
 - Manages storage
-

Docker Images

Read-only templates used to create containers.

Example:

```
None  
nginx  
ubuntu  
mysql
```

Docker Containers

Running instances of images.

Example:

```
None  
docker run nginx
```

creates a container from the nginx image.

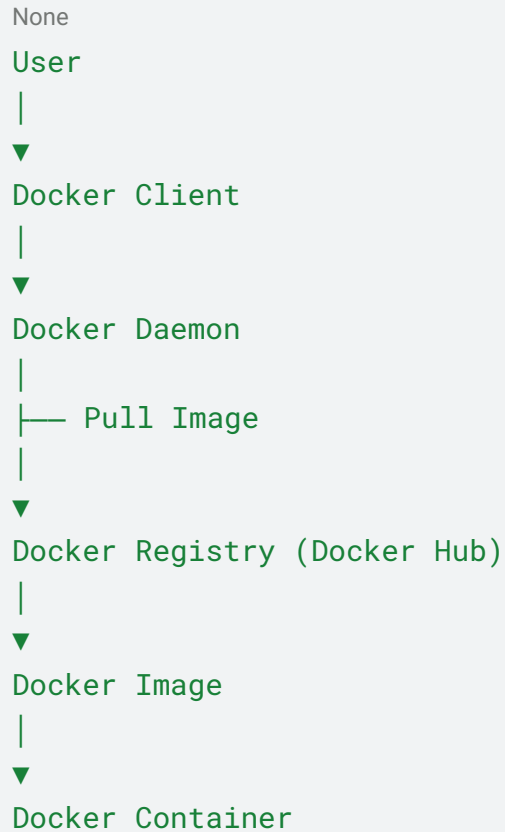
Docker Registry

Stores Docker images.

Examples:

- Docker Hub
 - Private Registries
-

Docker Architecture Flow



Task 2: Install Docker

Windows

1. Download Docker Desktop.
2. Install it.
3. Restart system if required.
4. Start Docker Desktop.

Linux (Ubuntu)

Update packages:

None

```
sudo apt update
```

Install Docker:

None

```
sudo apt install docker.io -y
```

Start Docker:

None

```
sudo systemctl start docker
```

Enable at boot:

None

```
sudo systemctl enable docker
```

Check status:

None

```
sudo systemctl status docker
```

```
No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-20-168:~$ sudo systemctl start docker
ubuntu@ip-172-31-20-168:~$ sudo systemctl enable docker
ubuntu@ip-172-31-20-168:~$ sudo systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-06-15 03:54:24 UTC; 1min 14s ago
 Invocation: d389f1bca5b4491db80c7cea2690550b
   TriggeredBy: ● docker.socket
      Docs: https://docs.docker.com
    Main PID: 2177 (dockerd)
       Tasks: 7
      Memory: 25.9M (peak: 26.6M)
         CPU: 32ms
    CGroup: /system.slice/docker.service
            └─2177 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

Jun 15 03:54:23 ip-172-31-20-168 dockerd[2177]: time="2026-06-15T03:54:23.833446288Z" level=info msg="Restoring containers: start."
Jun 15 03:54:23 ip-172-31-20-168 dockerd[2177]: time="2026-06-15T03:54:23.874641188Z" level=info msg="Deleting nftables IPv4 rules" error="exit status 1"
Jun 15 03:54:23 ip-172-31-20-168 dockerd[2177]: time="2026-06-15T03:54:23.876850160Z" level=info msg="Deleting nftables IPv6 rules" error="exit status 1"
Jun 15 03:54:24 ip-172-31-20-168 dockerd[2177]: time="2026-06-15T03:54:24.232609852Z" level=info msg="Loading containers: done."
Jun 15 03:54:24 ip-172-31-20-168 dockerd[2177]: time="2026-06-15T03:54:24.240238701Z" level=info msg="Docker daemon" commit=29.1.3-0ubuntu4.1 containerd=sn
Jun 15 03:54:24 ip-172-31-20-168 dockerd[2177]: time="2026-06-15T03:54:24.240696688Z" level=info msg="Initializing buildkit"
Jun 15 03:54:24 ip-172-31-20-168 dockerd[2177]: time="2026-06-15T03:54:24.279785068Z" level=info msg="Completed buildkit initialization"
Jun 15 03:54:24 ip-172-31-20-168 dockerd[2177]: time="2026-06-15T03:54:24.295036255Z" level=info msg="Daemon has completed initialization"
Jun 15 03:54:24 ip-172-31-20-168 dockerd[2177]: time="2026-06-15T03:54:24.295227220Z" level=info msg="API listen on /run/docker.sock"
Jun 15 03:54:24 ip-172-31-20-168 systemd[1]: Started docker.service - Docker Application Container Engine.
lines 1-23/23 (END)
```

i-07bf56512987790a5 (test-server)

PublicIPs: 52.91.140.123 PrivateIPs: 172.31.20.168

Verify Installation

Run:

None

```
docker --version
```

```
ubuntu@ip-172-31-20-168:~$ docker --version
Docker version 29.1.3, build 29.1.3-0ubuntu4.1
ubuntu@ip-172-31-20-168:~$
```

Add your user to the Docker group

This is the standard approach. It allows you to run `docker` commands without prepending `sudo` every single time.

Step 1: Create the docker group (It usually exists already, but just in case):

Bash

None

```
sudo groupadd docker
```

Step 2: Add your current user to the docker group:

Bash

None

```
sudo usermod -aG docker $USER
```

Step 3: Activate the changes to your groups: Instead of logging completely out and back into your SSH session, you can run this command to apply the group changes immediately to your current terminal:

Bash

None

```
newgrp docker
```

```
ubuntu@ip-172-31-20-168:~$ docker run hello-world
permission denied while trying to connect to the docker API at unix:///var/run/docker.sock
ubuntu@ip-172-31-20-168:~$ sudo groupadd docker
groupadd: group 'docker' already exists
ubuntu@ip-172-31-20-168:~$ sudo usermod -aG docker $USER
ubuntu@ip-172-31-20-168:~$ newgrp docker
ubuntu@ip-172-31-20-168:~$ docker search hello-world
NAME                                DESCRIPTION                                STARS     OFFICIAL
hello-world                        Hello World! (an example of minimal Dockeriz... 2578      [OK]
rancher/hello-world                This container image is no longer maintained... 6
okteto/hello-world                  0
atlassian/hello-world              3
goharbor/hello-world               0
tutum/hello-world                  Image to test docker deployments. Has Apache... 91
dockercloud/hello-world            Hello World!                                  20
```

Step 4: Test it out!

Bash

None

```
docker run hello-world
```

```
ubuntu@ip-172-31-20-168:~$ docker search hello-world
NAME                                DESCRIPTION                                STARS     OFFICIAL
hello-world                        Hello World! (an example of minimal Dockeriz... 2578      [OK]
rancher/hello-world                This container image is no longer maintained... 6
okteto/hello-world                  0
atlassian/hello-world              3
goharbor/hello-world               0
tutum/hello-world                  Image to test docker deployments. Has Apache... 91
dockercloud/hello-world            Hello World!                                  20

ubuntu@ip-172-31-20-168:~$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:96498ffd522e70807ab6384a5c0485a79b9c7c08ca79ba08623edcad1054e62d
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.
```

Run Hello World

None

```
docker run hello-world
```

Expected output:

None

```
Hello from Docker!
```

What happened:

1. Docker checked for image locally.
 2. Image not found.
 3. Downloaded image from Docker Hub.
 4. Created container.
 5. Executed program.
 6. Displayed message.
 7. Container stopped.
-

Task 3: Run Real Containers

Run Nginx Container

None

```
docker run -d -p 8080:80 nginx
```

```
ubuntu@ip-172-31-20-168:~$ docker run -d -p 8080:80 nginx
27e49be40eae3822766b1b8e885467475d667bea8215efcc141fb773cc9ccdbb
ubuntu@ip-172-31-20-168:~$
```

Explanation:

None

<code>-d</code>	Detached mode
<code>-p</code>	Port mapping
<code>8080</code>	Host port
<code>80</code>	Container port

Check:

None

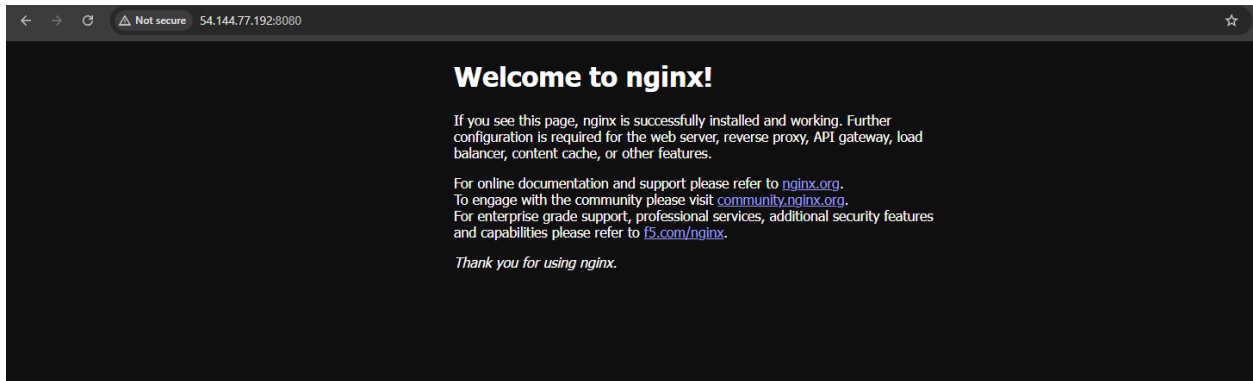
```
docker ps
```

```
ubuntu@ip-172-31-20-168:~$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED    STATUS    PORTS                               NAMES
27e49be40eae   nginx    "/docker-entrypoint..." 29 seconds ago    Up 28 seconds    0.0.0.0:8080->80/tcp, [::]:8080->80/tcp    loving_thompson
ubuntu@ip-172-31-20-168:~$
```

Open browser:

None

`http://EC2 PUBLIC IP:8080`



You should see:

Run Ubuntu Container Interactively

None

`docker run -it ubuntu`

Explanation:

None

`-i` Interactive
`-t` Terminal

```
ubuntu@ip-172-31-20-168:~$ docker run -it ubuntu
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
1c24335ddd46: Pull complete
6f5c5aa4e145: Pull complete
9bcf140d7f0f: Download complete
Digest: sha256:f3d28607ddd78734bb7f71f117f3c6706c666b8b76cbff7c9ff6e5718d46ff64
Status: Downloaded newer image for ubuntu:latest
root@639a9fd9d769:/# whoami
root
root@639a9fd9d769:/#
```

List Running Containers

None

`docker ps`

```
ubuntu@ip-172-31-20-168:~$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED    STATUS    PORTS                               NAMES
4011f42365d5   nginx    "/docker-entrypoint..." 25 minutes ago    Up 25 minutes    0.0.0.0:8080->80/tcp, [::]:8080->80/tcp    festive_bouman
ubuntu@ip-172-31-20-168:~$
```

List All Containers

None

`docker ps -a`

```
ubuntu@ip-172-31-20-168:~$ docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED    STATUS    PORTS                               NAMES
639a9fd9d769   ubuntu    "/bin/bash"             About a minute ago    Exited (0) 32 seconds ago                                clever_volhard
4011f42365d5   nginx    "/docker-entrypoint..." 25 minutes ago    Up 25 minutes    0.0.0.0:8080->80/tcp, [::]:8080->80/tcp    festive_bouman
27e49be40eae   nginx    "/docker-entrypoint..." About an hour ago    Exited (0) 54 minutes ago                                loving_thompson
921561a73e49   nginx    "/docker-entrypoint..." About an hour ago    Exited (0) About an hour ago                                eloquent_williams
40ea6f025d96   hello-world "/hello"                About an hour ago    Exited (0) About an hour ago                                reverent_austin
ubuntu@ip-172-31-20-168:~$
```

Stop Container

Find ID:

None

`docker ps`

Stop:

None

`docker stop <container_id>`

```
ubuntu@ip-172-31-20-168:~$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED    STATUS    PORTS                               NAMES
4011f42365d5   nginx    "/docker-entrypoint..." 26 minutes ago    Up 26 minutes    0.0.0.0:8080->80/tcp, [::]:8080->80/tcp    festive_bouman
ubuntu@ip-172-31-20-168:~$ docker stop 4011f42365d5
4011f42365d5
ubuntu@ip-172-31-20-168:~$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED    STATUS    PORTS    NAMES
ubuntu@ip-172-31-20-168:~$
```

Remove Container

None

```
docker rm <container_id>
```

Example:

None

```
docker rm abc123
```

Task 4: Explore Docker Features

Run Container in Detached Mode

None

```
docker run -d nginx
```

Output:

None

```
7d8f91c...
```

Difference:

- Terminal remains free.
 - Container runs in background.
-

Give Container a Custom Name

None

```
docker run -d --name mynginx nginx
```

Check:

None

`docker ps`

```
ubuntu@ip-172-31-20-168:~$ docker run -d --name mynginx nginx
18afaf07ab3e1f6e3a0baf147cc73a1c62692e2810bfd2e298791af0ca63e066
ubuntu@ip-172-31-20-168:~$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS        NAMES
18afaf07ab3e   nginx    "/docker-entrypoint..." 4 seconds ago  Up 3 seconds  80/tcp       mynginx
ubuntu@ip-172-31-20-168:~$
```

Port Mapping

Run:

None

`docker run -d -p 9090:80 nginx`

sg-012cf91c3e00762f8 - launch-wizard-7

✓ Inbound security group rules successfully modified on security group (sg-012cf91c3e00762f8 | launch-wizard-7)

Details

sg-012cf91c3e00762f8 - launch-wizard-7

Actions

Details

Security group name	Security group ID	Description	VPC ID
launch-wizard-7	sg-012cf91c3e00762f8	launch-wizard-7 created 2026-06-15T03:50:30.828Z	vpc-085f4d62e2df7c23e
Owner	Inbound rules count	Outbound rules count	
769369611777	2 Permission entries	1 Permission entry	

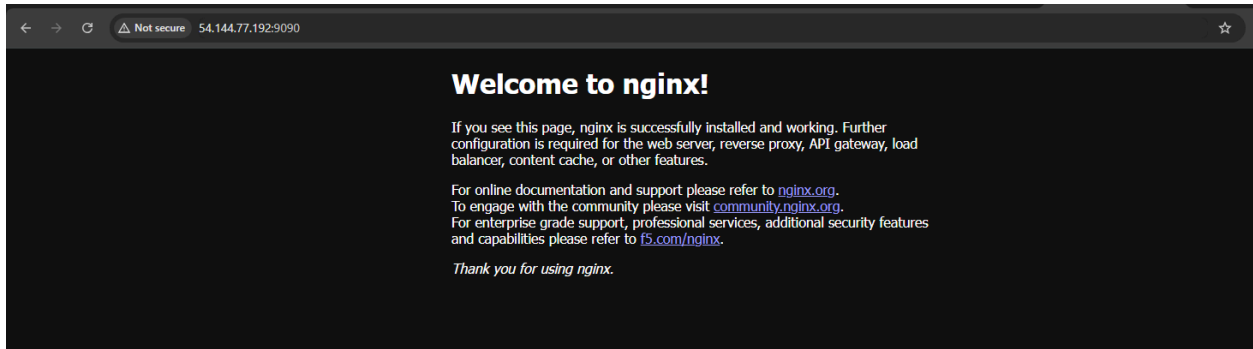
Inbound rules Outbound rules Sharing VPC associations Related resources - new Tags

Inbound rules (2)

Search

☐	Name	Security group rule ID	IP version	Type	Protocol	Port range	Source
☐	-	sgr-00bedc4d73adc38c7	IPv4	Custom TCP	TCP	9090	0.0.0.0/0
☐	-	sgr-04808888f818aa1a7	IPv4	SSH	TCP	22	0.0.0.0/0

```
ubuntu@ip-172-31-20-168:~$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                                     NAMES
ce5e6c9bfa0b   nginx    "/docker-entrypoint..." 11 seconds ago  Up 11 seconds  0.0.0.0:9090->80/tcp, [::]:9090->80/tcp  parthavinginx
ubuntu@ip-172-31-20-168:~$
```



View Logs

None

```
docker logs mynginx
```

Follow live logs:

None

```
docker logs -f mynginx
```

Execute Commands Inside Running Container

Open shell:

None

```
docker exec -it mynginx bash
```

```
ubuntu@ip-172-31-20-168:~$ docker exec -it parthavinginx bash
root@ce5e6c9bfacb:/#
```

If bash unavailable:

None

```
docker exec -it mynginx sh
```

Run:

```
None  
ls  
pwd  
hostname
```

Exit:

```
None  
exit
```

Final Verification Commands

Run all of these successfully:

```
None  
docker --version  
  
docker run hello-world  
  
docker run -d -p 8080:80 nginx  
  
docker run -it ubuntu  
  
docker ps  
  
docker ps -a  
  
docker logs mynginx  
  
docker exec -it mynginx bash  
  
docker stop mynginx
```

```
docker rm mynginx
```

Project 1: Dockerfile flask-app

1. Git clone
2. Rm Dockerfile(as already there in project, i made mine)
3. Vim Dockerfile

```
FROM python:3.14

WORKDIR /app

COPY . .

RUN pip install -r requirements.txt

CMD ["python", "run.py"]
```

- ## 4. Build

docker build -t flask-app .

5. Run

docker run -d -p 5122:80 flask-app

```
ubuntu@ip-172-31-81-141:~/flask-app-ecs$ docker run -d -p 5122:80 flask-app
c3676ee2412965074947329e939940c05cd2025e5736d591ea2ed4e8f831955a
```

6. In AWS sg,

sg-02a5f8ec6d07a7218 - launch-wizard-9

Inbound security group rules successfully modified on security group (sg-02a5f8ec6d07a7218 | launch-wizard-9)

Details

sg-02a5f8ec6d07a7218 - launch-wizard-9 Actions

Details			
Security group name launch-wizard-9	Security group ID sg-02a5f8ec6d07a7218	Description launch-wizard-9 created 2026-06-15T15:18:05.303Z	VPC ID vpc-085f4d62e2df7c23e12
Owner 769369611777	Inbound rules count 2 Permission entries	Outbound rules count 1 Permission entry	

Inbound rules Outbound rules Sharing VPC associations Related resources - new Tags

Inbound rules (2) Manage tags Edit inbound rules

Search

	Name	Security group rule ID	IP version	Type	Protocol	Port range
☐	-	sgr-0572b5bd1809c5b47	IPv4	SSH	TCP	22
☐	-	sgr-08a7aaa4fe4f57c63	IPv4	Custom TCP	TCP	5122

```
ubuntu@ip-172-31-81-141:~$ git clone https://github.com/LondheShubham153/flask-app-ecs.git
Cloning into 'flask-app-ecs'...
remote: Enumerating objects: 62, done.
remote: Counting objects: 100% (32/32), done.
remote: Compressing objects: 100% (23/23), done.
remote: Total 62 (delta 18), reused 9 (delta 9), pack-reused 30 (from 1)
Receiving objects: 100% (62/62), 13.89 KiB | 1.39 MiB/s, done.
Resolving deltas: 100% (18/18), done.
ubuntu@ip-172-31-81-141:~$ ls
flask-app-ecs
ubuntu@ip-172-31-81-141:~$ cd flask-app-ecs/
ubuntu@ip-172-31-81-141:~/flask-app-ecs$ ls
Dockerfile Dockerfile-multi README.md app.py requirements.txt run.py templates
ubuntu@ip-172-31-81-141:~/flask-app-ecs$ rm Dockerfile
ubuntu@ip-172-31-81-141:~/flask-app-ecs$ ls
Dockerfile-multi README.md app.py requirements.txt run.py templates
ubuntu@ip-172-31-81-141:~/flask-app-ecs$ vim Dockerfile
ubuntu@ip-172-31-81-141:~/flask-app-ecs$ docker build -t flask-app .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
             Install the buildx component to build images with BuildKit:
             https://docs.docker.com/go/buildx/

Sending build context to Docker daemon 95.74kB
Step 1/5 : FROM python:3.14
3.14: Pulling from library/python
960d9f93a2b3: Pulling fs layer
442cca122a8b: Pulling fs layer
ef1b08ddd59d: Pulling fs layer
018f0d9721ad: Pulling fs layer
2fc17a7c22c0: Pulling fs layer
```

i-0e24c87d2a50fe107 (web-server)